

Cloud native temperature excursion and compliance platform

SHAAMAK	Pseudocode of Simulation Script
RIYA	Kafka(creating topics,partitions)
RUSSEL	Kubernetes overview
PRANAV	Logistics manager , Warehouse Receiver
RONAK	Compliance officer, System Administrator

KAFKA

- Studied basics of Kafka architecture (topics, producers, consumers)
- Learned how to create Kafka topics for communication
- Understood partitions and how they improve performance
- Explored how Kafka enables asynchronous communication between services
- **AND ALSO EXPLORED DOCKER TOPICS**

Work to be done next week

- Test multiple consumers reading from partitions
- Understand offset management (how Kafka tracks messages)

Simulation Script Pseudocode

- Designed simulation flow to fetch sensors and run one worker per sensor
- Implemented continuous loop to generate temperature and location data every 10 seconds
- Added real-world scenarios like fridge failure and network loss using random conditions
- Built logic to buffer data during no signal and send it later to Apache Kafka


```
1  # — PART 1 : Start the service (runs once) —————
2
3  START:
4      Connect to Kafka
5      Ask Shipment API "give me all in-transit sensors"
6
7      FOR EACH sensor in that list:
8          Launch a background worker ← all run simultaneously
9
10     Keep the program alive forever
11
12
13  # — PART 2 : Worker loop (one per sensor, runs forever) —
14
15  LOOP forever:
16
17      # Step 1 — Decide the state
18      IF random_chance (2%) → state = FRIDGE_BROKE
19      IF random_chance (5%) → state = NO_SIGNAL
20      ELSE → state = NORMAL
```

```
21
22     # Step 2 – Generate a temperature reading
23     IF state == FRIDGE_BROKE → temperature += 2.0°C (breach)
24     ELSE → temperature += tiny random drift
25
26     # Step 3 – Build the message
27     payload = { timestamp, sensor_id, gps, temperature }
28
29     # Step 4 – Send or save
30     IF state == NO_SIGNAL → save to local buffer
31     IF just returned to signal → flush entire buffer to Kafka
32     IF state == NORMAL → publish to Kafka immediately
33
34     # Step 5 – Wait, then repeat
35     WAIT 10 seconds → go back to Step 1
```

Work to be done next week

- Writing the script in Python using asyncio for concurrent sensor workers
- aiokafka library to publish messages to the telemetry-events Kafka topic

Kubernetes:

- It allows running multiple containers
- Provides scalability
- Ensures high availability

Simple Implementation:

```
PS C:\Users\russe> kubectl get all
NAME                                READY   STATUS    RESTARTS   AGE
pod/mongo-deployment-54f4bdd58-hhf2h 1/1     Running   0           117s
pod/webapp-deployment-5766fd95c7-b7n6h 1/1     Running   0           76s

NAME                                TYPE               CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/kubernetes                  ClusterIP          10.96.0.1       <none>        443/TCP          3m10s
service/mongo-service               ClusterIP          10.104.41.213   <none>        27017/TCP        117s
service/webapp-service              NodePort           10.108.103.56    <none>        3000:30100/TCP   76s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/mongo-deployment    1/1     1             1           117s
deployment.apps/webapp-deployment    1/1     1             1           76s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/mongo-deployment-54f4bdd58 1         1         1       117s
replicaset.apps/webapp-deployment-5766fd95c7 1         1         1       76s
```

```
replicaset.apps/webapp-deployment-5766fd95c7 1 1 1 76s
PS C:\Users\russe> kubectl get configmap
NAME      DATA  AGE
kube-root-ca.crt 1      3m41s
mongo-config      1      2m24s
```

```
mongopassword
PS C:\Users\russe> minikube service webapp-service

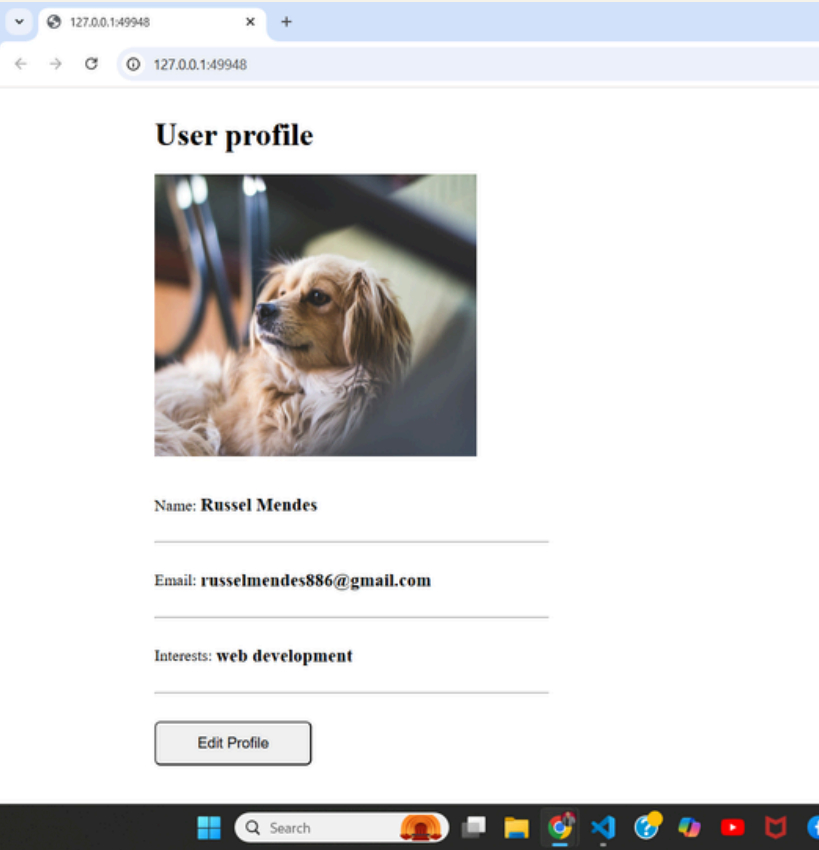
NAMESPACE   NAME       TARGET PORT  URL
default     webapp-service 3000         http://192.168.49.2:30100

Starting tunnel for service webapp-service.

NAMESPACE   NAME       TARGET PORT  URL
default     webapp-service      http://127.0.0.1:49948

Opening service default/webapp-service in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

UI:



Database:

```
test> use my-db
switched to db my-db
my-db> show collections
users
my-db> db.users.find().pretty()
[
  {
    _id: ObjectId('69c659b2303047d1012de699'),
    userid: 1,
    email: 'russelmendes886@gmail.com',
    interests: 'web development',
    name: 'Russel Mendes'
  }
]
my-db> |
```

```
PS C:\Users\russe> kubectl get secret
NAME          TYPE      DATA  AGE
mongo-secret  Opaque    2       24m
```

Works to be done next week

How to implement modules as microservices using Kubernetes

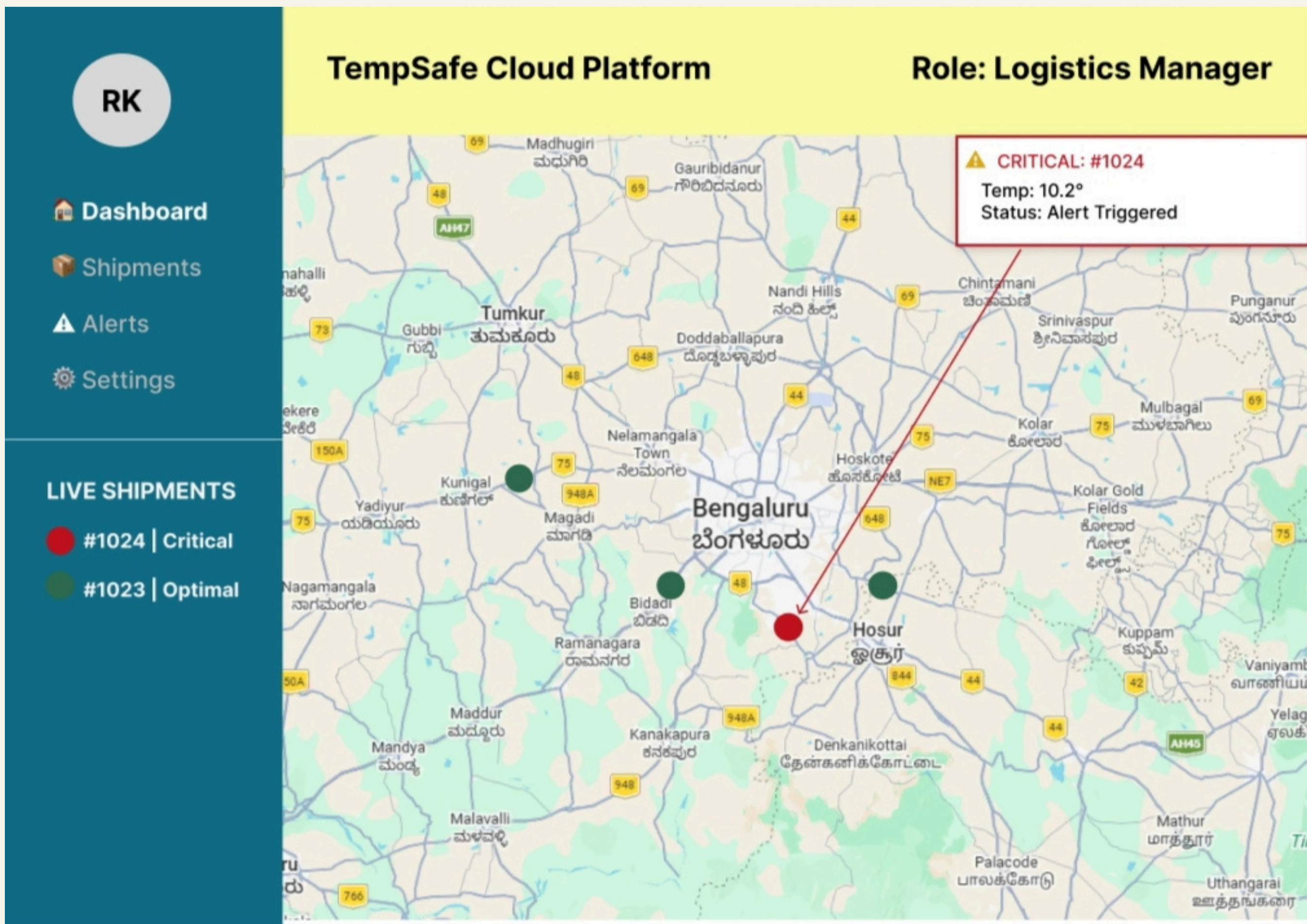
UI Design

- **Introduction**
- **Icons**

Dashboard , Shipments, Alerts and Settings

- **Live shipments**
- **The Profile**

Logistics Manager :



Warehouse Receiver :

RK

 Dashboard

 Shipments

 Alerts

 Settings

LIVE SHIPMENTS

 #1024 | Critical

 #1023 | Optimal

TempSafe Cloud Platform

Role: Warehouse Receiver

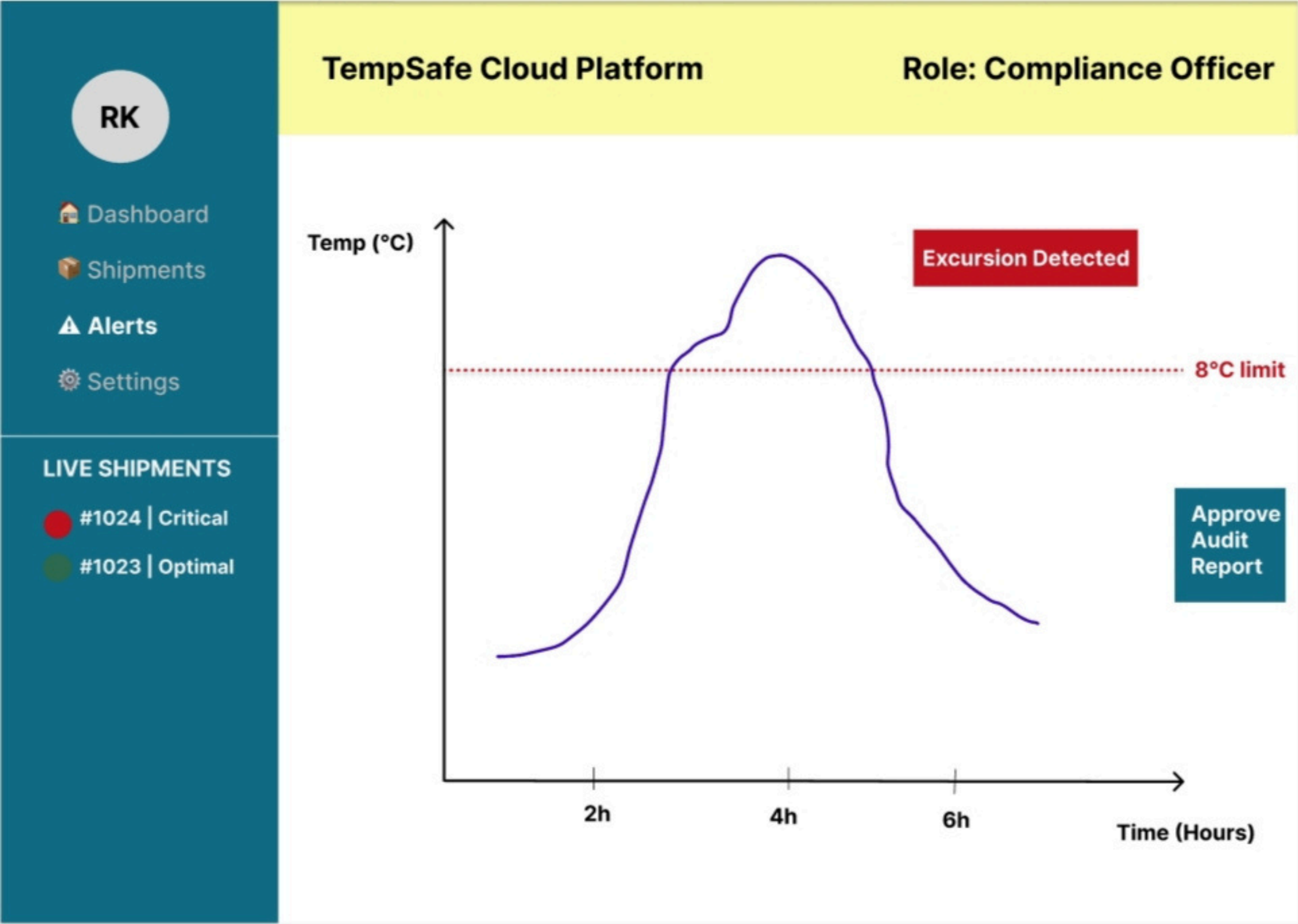
 Start Loading

 Mark as Shipped

 Unload

Note: Manual temperature override is disabled. All sensor data is locked to the blockchain/immutable log

Compliance officer :



System Administrator :

RK

Dashboard

Shipments

Alerts

Settings

LIVE SHIPMENTS

#1024 | Critical

#1023 | Optimal

TempSafe Cloud Platform

Role: System Administrator

Access Control List

Pranav (Admin) — FULL_SYSTEM

Ronak (Receiver) — TRIP_STATUS_ONLY

Suresh (Compliance) — AUDIT_REPORT_READ

SYSTEM INTEGRITY STATUS

Database: Encrypted & Immutable

Raw Logs: Read-Only

Constraint: Admin override disabled for sensor data

Works to be done next week

- **Component Architecture**
- **Navigation & Routing**
- **API & Backend Connectivity**